

Caso Telco Churn

En este caso de estudio vamos a trabajar con el dataset **telco_churn.csv**, que contiene información de clientes de una empresa de telecomunicaciones.

El desafío es **entrenar distintos modelos de clasificación** y seleccionar aquel que tenga **mayor poder predictivo** para anticipar la probabilidad de abandono de un cliente (*churn*).

El objetivo es lograr un modelo capaz de **predecir si un cliente dejará de utilizar el servicio** a partir de las siguientes variables:

- **tenure_months**: meses que el cliente lleva con la empresa.
- **monthly_charges**: importe mensual promedio que paga.
- **total_services**: cantidad de servicios activos (internet, TV, telefonía, etc.).
- **support_calls_last90d**: llamadas al soporte en los últimos 90 días.
- **late_payments_12m**: cantidad de pagos atrasados en los últimos 12 meses.
- **downtime_hrs_6m**: horas acumuladas de caída del servicio en los últimos 6 meses.
- **contract**: tipo de contrato (*month-to-month* < *one-year* < *two-year*).
- **support_tier**: nivel de soporte contratado (*basic* < *plus* < *premium*).
- **promo_status**: estado de promociones (*never* < *past* < *current*).
- **internet**: tipo de conexión principal (*dsl*, *fiber*, *none*).
- **payment_method**: forma de pago (*electronic check*, *credit card*, *bank transfer*, *mailed check*).

Dinámica de la clase

1. Exploración inicial

Revisar el análisis exploratorio ya disponible en el siguiente script:

[Colab – Análisis Exploratorio](#)

- ¿Cuáles son los *priors* de este dataset?
 - Si usamos un modelo trivial que siempre predice 0, ¿qué *accuracy* esperaríamos?
 - ¿Qué métrica conviene utilizar en este caso para evaluar la performance de un modelo de clasificación?
-

2. Primer modelo: árbol de decisión por defecto

- Entrenar un árbol de clasificación con los parámetros por defecto de *sklearn*.
 - Calcular la métrica elegida y compararla con el *accuracy*.
 - Responder: ¿Están conformes con los resultados? ¿Qué ajustes serían necesarios para mejorarlos?
 - Graficar el árbol entrenado.
-

3. Selección de hiperparámetros con Cross-Validation

- Usar **GridSearchCV** o **RandomizedSearchCV** (a elección) con la grilla provista.
- Entrenar el modelo con los hiperparámetros seleccionados.
- Calcular la métrica elegida y compararla nuevamente con el *accuracy*.
- Responder: ¿Qué observan? ¿Cuánto mejora el modelo al elegir hiperparámetros por CV?
- Graficar el árbol resultante.

Grilla para gridsearch:

```
param_grid = {  
    "clf__class_weight": [None, "balanced"],  
    "clf__max_depth": [None, 6, 10, 14, 18, 24],  
    "clf__min_samples_split": [2, 20, 60, 120, 200],  
    "clf__min_samples_leaf": [1, 10, 30, 60, 100],  
    "clf__ccp_alpha": [0.0, 0.0005, 0.001, 0.005, 0.01]  
}
```

Grilla para randomized search:

```
param_dist = {  
    "clf__class_weight": [None, "balanced"],  
    "clf__max_depth": randint(3, 30),  
    "clf__min_samples_split": randint(2, 300),  
    "clf__min_samples_leaf": randint(1, 120),  
    "clf__ccp_alpha": uniform(0.0, 0.02),  
}
```

4. Extensión (opcional)

- Entrenar otros modelos vistos en clase (KNN, regresión logística, etc.).
- Comparar resultados.
- Responder: ¿Qué modelo seleccionarían como “mejor” y por qué?